

Q1. Real-Time Detection System

A surveillance system evaluates three candidate techniques — Viola-Jones, YOLO, and Deep Learning — for real-time face/object detection. The comparison below weighs detection accuracy against processing speed to identify the method best suited for strict latency constraints.

Method	Accuracy	Speed
Viola-Jones	78%	Fast
YOLO	92%	Moderate
Deep Learning	95%	Slow

Figure 1: Accuracy vs Speed Trade-off Across Detection Methods

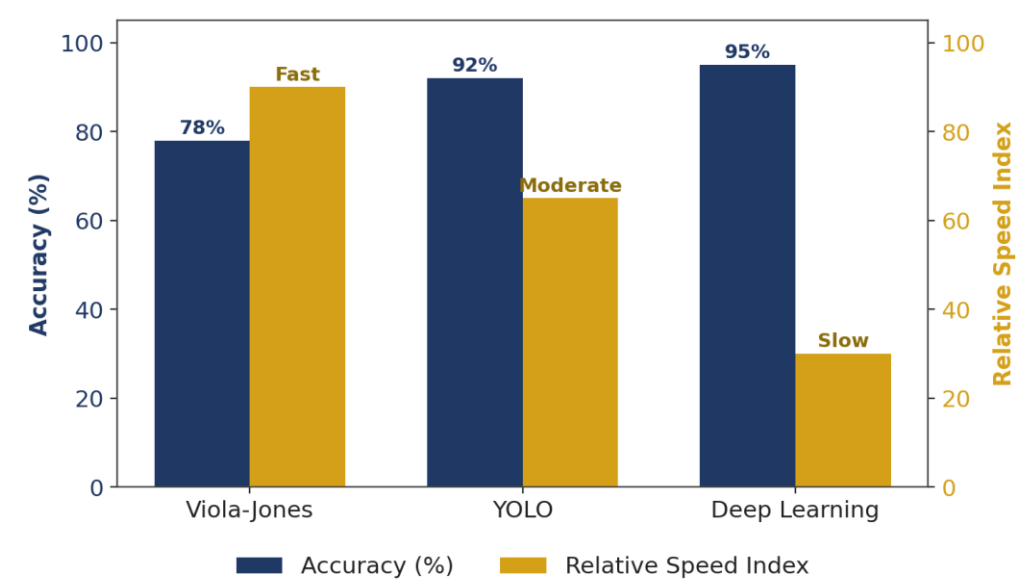


Figure 1: Accuracy rises with model complexity while processing speed falls — YOLO occupies the balanced middle ground.

Comparative Analysis

- **Viola-Jones:** Fastest inference (Haar-cascade based), but only 78% accuracy — prone to false negatives with pose/lighting variation, unsuitable where reliability matters.
- **YOLO:** Single-pass CNN detector giving 92% accuracy at moderate speed — a strong middle ground between the two extremes.

- **Deep Learning (e.g. Faster R-CNN/deep backbones):** Highest accuracy (95%) but slow inference, making it unsuitable for hard real-time constraints without powerful hardware (GPU/edge accelerators).

Evaluation and Recommendation

For a surveillance system operating under strict latency constraints while still requiring acceptable accuracy, YOLO is the recommended choice. It delivers a 92% accuracy — only 3 points below the slowest, most accurate Deep Learning model — while running at moderate speed, close to real-time frame rates (typically 30+ FPS on modern hardware). Viola-Jones, though fastest, sacrifices too much accuracy for a security-critical application where missed detections carry real consequences. Deep Learning models, while most accurate, introduce latency that can cause frame drops and delayed alerts in live surveillance feeds, and typically require expensive, dedicated hardware to run in real time.

Deployment consideration: If the system runs on edge devices (CCTV-embedded chips, IoT cameras) with limited compute, YOLO's single-shot architecture is far more deployable than deep multi-stage networks. Deep Learning models remain preferable only when latency is not safety-critical and GPU clusters are available (e.g., forensic post-event analysis rather than live monitoring).

Q2. Robust Object Detection

Real-world surveillance rarely operates under ideal conditions. This section compares how each method performs when faced with occlusion and low-light environments, in addition to normal conditions.

Method	Normal	Occlusion	Low Light
Viola-Jones	80%	60%	55%
YOLO	90%	75%	70%
Deep Learning	95%	85%	80%

Figure 2: Robustness Across Normal, Occlusion, and Low-Light Conditions

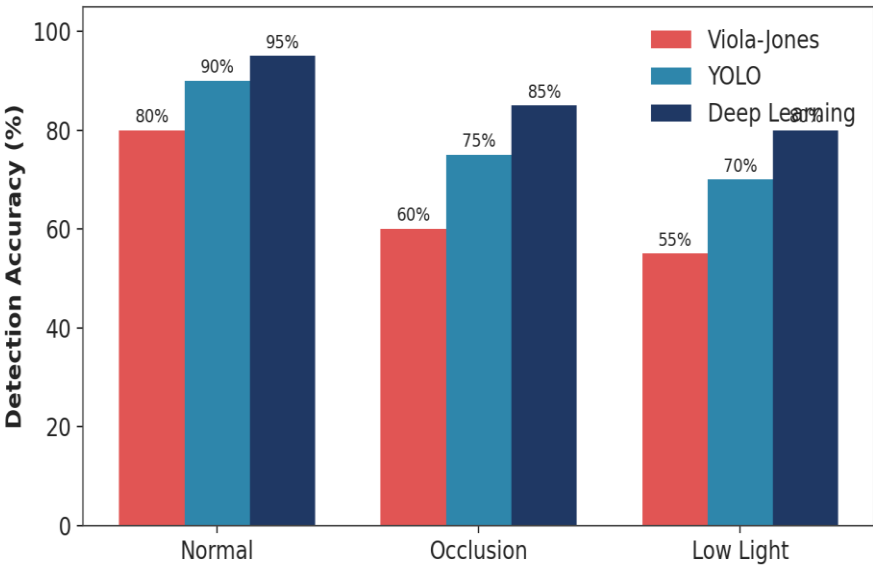


Figure 2: All methods degrade under occlusion and low light, but Deep Learning retains the smallest performance drop.

Comparative Analysis

- **Viola-Jones:** Degrades sharply under occlusion (−20%) and low light (−25%) because it depends on rigid Haar-like features that assume unobstructed, well-lit frontal patterns.
- **YOLO:** More resilient (−15% occlusion, −20% low light) due to learned convolutional features that generalize better across appearance variation, though performance still declines meaningfully.
- **Deep Learning:** Most robust (−10% occlusion, −15% low light) — deeper hierarchical feature extraction captures partial and low-contrast patterns more effectively than shallower models.

Evaluation and Trade-offs

Deep Learning provides the best overall robustness across all three conditions, retaining the highest absolute accuracy (80%) even in the worst-case low-light scenario. This makes it the strongest candidate where environmental conditions are unpredictable — e.g., outdoor or 24-hour surveillance. However, this robustness comes at the cost of computational overhead and slower inference (as shown in Q1), so it suits applications where accuracy under adverse conditions outweighs the need for instant response.

YOLO offers a reasonable compromise: it does not match Deep Learning's robustness but still comfortably outperforms Viola-Jones in every condition, while remaining fast enough for real-time use. Viola-Jones should be limited to controlled, well-lit environments with minimal occlusion (e.g., indoor access-control checkpoints) where its speed advantage can be exploited safely.

Q3. Optical Flow Analysis

Two optical flow methods are considered for motion tracking. Method A produces stable motion vectors but misses small motions; Method B detects fine motion but produces noisy vectors. Since no numeric accuracy table is given, the comparison below is qualitative, scored on a relative 1–10 scale for interpretability.

Criterion	Method A	Method B
Motion Sensitivity (detects small movements)	Low	High
Vector Stability (consistency over time)	High	Low
Noise Sensitivity	Low (robust)	High (noisy)
Best suited for	Slow / large-scale motion	Fast / fine-grained motion

Figure 3: Qualitative Comparison of Optical Flow Methods

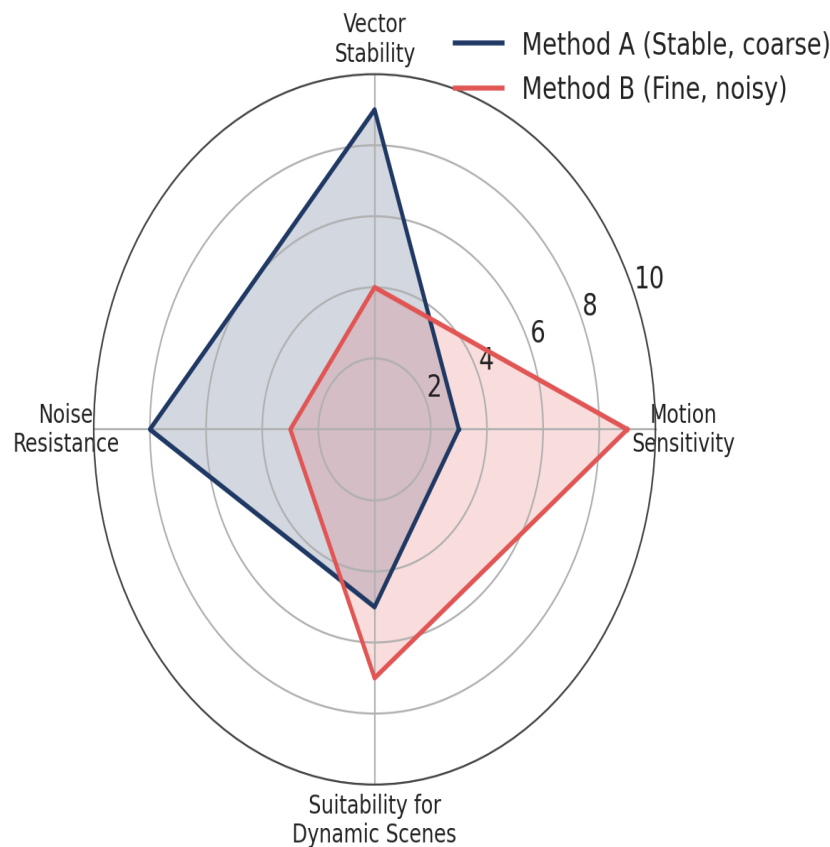


Figure 3: Method A trades sensitivity for stability; Method B trades stability for sensitivity — a classic precision-vs-reliability trade-off.

Comparative Analysis

- **Method A:** Produces smooth, temporally consistent motion vectors, making it reliable for tracking large or steady movements, but its low sensitivity means small or subtle motions (e.g., a hand gesture, a distant object) are missed entirely.
- **Method B:** Captures fine-grained motion that Method A would overlook, but the noisy vectors it produces can trigger false positives or unstable tracking trajectories, especially in cluttered dynamic scenes.

Evaluation and Recommendation

For motion tracking in dynamic scenes — where multiple objects move at varying speeds and directions — neither method alone is fully sufficient, but Method A is the more practically suitable base method. Stability is critical in dynamic scenes because noisy vectors compound over consecutive frames, causing tracked object trajectories to drift or jitter, which is far more disruptive than occasionally missing a very small motion. In applications like surveillance and pedestrian/vehicle tracking, a stable-but-slightly-conservative estimate is more usable than a sensitive-but-erratic one.

In practice, the two are often combined: Method A supplies a stable baseline trajectory, while Method B (or a noise-filtered version of it, e.g., smoothed with a Kalman filter) supplements fine-motion detection. This hybrid approach balances accuracy, stability, and noise sensitivity better than either method used in isolation.

Q4. Model Generalization and Overfitting

Two trained models are compared on training versus testing accuracy to assess how well each generalizes to unseen data.

Model	Training Accuracy	Testing Accuracy	Accuracy Gap
Model A	98%	70%	28%
Model B	90%	88%	2%

Figure 4: Training vs Testing Accuracy (Generalization Gap)

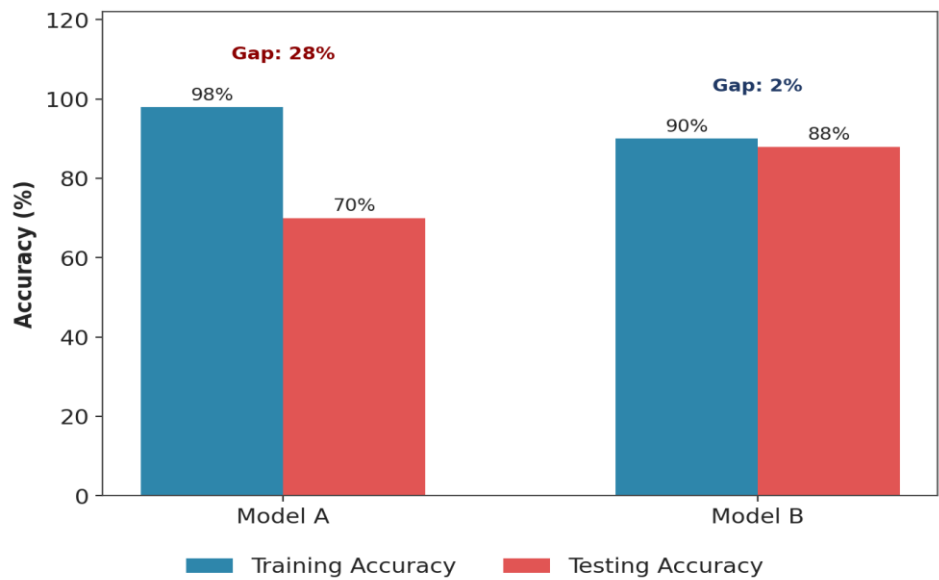


Figure 4: Model A's large train-test gap signals overfitting; Model B generalizes far more consistently.

Comparative Analysis

- **Model A:** Achieves near-perfect training accuracy (98%) but testing accuracy drops sharply to 70% — a 28-point gap. This large gap is a classic signature of overfitting: the model has memorized training-set patterns, including noise, rather than learning generalizable features.
- **Model B:** Slightly lower training accuracy (90%) but testing accuracy of 88%, only a 2-point gap. This small, consistent gap indicates the model has learned genuinely generalizable patterns rather than memorizing the training set.

Evaluation and Recommendation

Model B is clearly more reliable for real-world deployment. Real-world/production data is essentially "unseen" data from the model's perspective, so testing accuracy — not training accuracy — is the true indicator of expected field performance. Model A's 70% testing accuracy suggests it would perform unpredictably and often poorly once deployed, despite its impressive training scores. Model B's balanced 90%/88% accuracy demonstrates strong generalization ability and low overfitting risk, making its performance far more trustworthy and consistent in production.

Root causes to address for Model A (if it must be salvaged) include reducing model complexity, adding regularization (dropout, L2 weight decay), increasing training data diversity, and using early stopping or cross-validation during training. In its current state, however, Model B should be selected for deployment.

Q5. System Performance under Constraints

Three models are compared on accuracy, latency, and cost to identify the best balance for a real-time application with practical deployment constraints.

Model	Accuracy	Latency	Cost
Model A	88%	50 ms	Low
Model B	92%	80 ms	Moderate
Model C	95%	120 ms	High

Figure 5: Accuracy, Latency, and Cost Trade-off

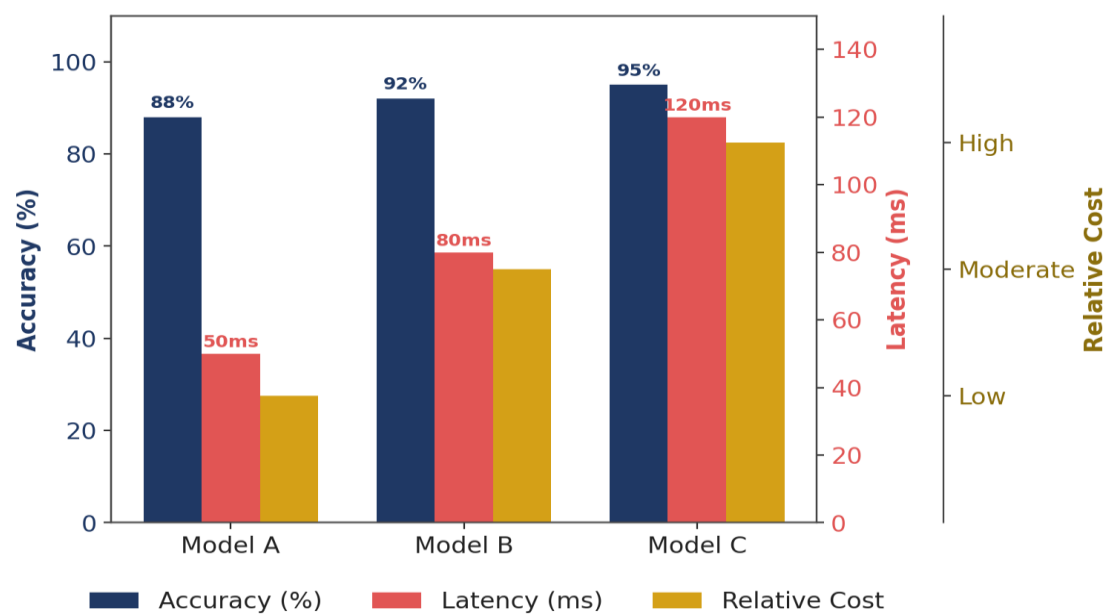


Figure 5: Accuracy rises with latency and cost — Model B sits at the inflection point of the trade-off curve.

Comparative Analysis

- **Model A:** Fastest (50 ms) and cheapest to run, but the lowest accuracy (88%) of the three — suitable only where speed/cost dominate and moderate accuracy is tolerable.
- **Model B:** Balanced profile — 92% accuracy at 80 ms latency and moderate cost. It captures most of the accuracy gain from A→C while adding only 30 ms over Model A, at a manageable cost increase.

- **Model C:** Highest accuracy (95%) but more than double Model A's latency (120 ms) and the highest cost — appropriate only when accuracy is paramount and latency/cost constraints are relaxed.

Evaluation and Recommendation

Model B provides the best overall balance for a real-time application. Its accuracy improvement over Model A (+4 points) is achieved at a modest latency cost (+30 ms), whereas Model C's further accuracy gain over Model B (+3 points) costs a disproportionate +40 ms in latency and a jump to high cost — a case of diminishing returns. For most real-time systems, an 80 ms response time is still comfortably within interactive/near-real-time thresholds (well under the ~100–200 ms range users perceive as immediate), making Model B's latency acceptable while its accuracy remains close to the best available option.

Deployment guidance: Choose Model A only for extremely latency-sensitive, low-budget deployments (e.g., high-volume edge devices) where a few accuracy points can be sacrificed. Choose Model C only for offline, batch, or safety-critical analysis where cost and delay are secondary to correctness. Model B is the recommended default for live, cost-conscious, real-time deployment.